

Predicción de Etapas del Sueño sobre Señales Fisiológicas capturadas con SmartWatch

Agustín Adrián Pereyra
Departamento de Ingeniería
Universidad de San Andrés
Buenos Aires, Argentina
apereyra@udesa.edu.ar

Agustín Patruno
Departamento de Ingeniería
Universidad de San Andrés
Buenos Aires, Argentina
apatruno@udesa.edu.ar

Resumen—La estadificación del sueño (*sleep staging*) es clave para el diagnóstico de trastornos del sueño pero depende de la polisomnografía (caro e incómodo). Abordamos el problema como una clasificación multiclase de etapas del sueño a partir únicamente de señales fisiológicas medidas por un SmartWatch con la clasificación de un experto como *ground-truth*. Comparamos una serie de modelos teniendo como resultado a una BiLSTM con features extraídas manualmente de las señales como modelo más óptimo, con métricas ($F1_{Macro} \approx 0.57$, $\kappa \approx 0.46$ frente al experto), superando en ~ 12 puntos de κ a los *baselines* (modelos tabulares simples) por época, lo que confirma que el contexto inter-época es el factor determinante.

Index Terms—sleep staging, recurrent neural networks, heart rate variability, accelerometry, EEG, deep learning

I. INTRODUCCIÓN

Frente a la polisomnografía —costosa y confinada al laboratorio—, los dispositivos *SmartWatch* son capaces de registrar IHR (*Instantaneous Heart Rate*) y acelerometría de forma continua y no invasiva, habilitando el monitoreo domiciliario.

La *estadificación del sueño* (*sleep staging*) —asignar una etapa a cada época de 30 s de registro, según el estándar clínico AASM— es central para el diagnóstico de trastornos como la apnea o el insomnio, pero hoy depende de la polisomnografía y de la anotación manual de un experto, lo que la vuelve cara e incómoda. Lograrla a partir de señales de muñeca abriría la puerta a un monitoreo accesible y prolongado en el tiempo.

Abordamos la tarea como un problema de **clasificación multiclase supervisada**. La entrada de nuestro algoritmo es la IHR y la acelerometría triaxial de una época de 30 s (o su secuencia a lo largo de la noche), y utilizamos modelos de aprendizaje automático desde un *baseline* tabular (XGBoost, MLP) hasta modelos secuenciales que modelan la dependencia temporal entre épocas (un BiLSTM sobre las características tabulares y un híbrido CNN—BiLSTM que aprende directamente de la señal cruda) para predecir la etapa del sueño. Como referencia se emplean las etiquetas de un experto (*ground-truth*) y una referencia secundaria (y como nuestra competencia directa) las etiquetas de un dispositivo “Dreem”. El eje del trabajo es comparar esta progresión de modelos para cuantificar el aporte del **contexto temporal inter-época** —la dependencia entre etapas contiguas— frente a la clasificación de cada época de forma aislada.

II. CONJUNTO DE DATOS Y CARACTERÍSTICAS

PhysioNet BIDSleep [1] recopila señales fisiológicas tales como la frecuencia cardíaca instantánea (IHR) y tres ejes de acelerometría medidas con un Apple Watch Series 6 a lo largo de 253 noches de 47 adultos sanos. Cada lapso de 30 s se encuentra etiquetado con una etapa del sueño por dos fuentes diferentes: *expert* es la clasificación que realiza un especialista/técnico en polisomnografía siguiendo el estándar AASM¹, mientras que *dreem* es la clasificación automática que realiza un dispositivo Dreem 2 Headband basado en EEG (electroencefalograma). Las etapas etiquetadas son:

0. Wake (Despierto)

1. **N1** (Sueño Ligero): etapa más superficial y la entrada al sueño. Suele ser la clase más difícil de detectar dado que es breve e inestable.
2. **N2** (Sueño Ligero consolidado): sujeto dormido de forma estable. Es la etapa más frecuente.
3. **N3** (Sueño profundo): el sueño más reparador. En él cuesta mucho despertar al paciente. Debería mostrar IHR mínimo y cuerpo inmóvil.
4. **REM** (*Rapid Eye Movement*): también llamado “sueño paradójico”. El EEG se parece al de vigilia pero el paciente está profundamente dormido, involucrando IHR irregular.

La distribución de clases (label expert) está fuertemente desbalanceada: N_2 (37.5 %) y REM (25.9 %) concentran la mayor parte de las épocas, seguidas de N_3 (18.5 %) y Wake (11.0 %), mientras que N_1 es marginal (7.1 %). Este desbalance justifica evaluar con $F1_{Macro}$, que pondera cada clase por igual, así como la utilización de funciones de costo ponderadas por clase en los modelos finales.

II-A. Procesamiento de la Señal

Realizamos una ardua y estricta limpieza de las señales dado que traían múltiples problemas:

- **Desalineación temporal:** el inicio de grabación viene en hora local (*America/New_York*) mientras que IHR y

¹Reglas de la *American Academy of Sleep Medicine*, el consenso clínico que define cómo puntuar las etapas del sueño a partir de la polisomnografía: divide el registro en épocas de 30 s y asigna a cada una una etapa (Wake, N1, N2, N3, REM) según criterios estandarizados sobre las señales de EEG, EOG y EMG.

acelerometría están en Unix UTC. Se corrige convirtiendo el timestamp de inicio de grabación con su zona horaria antes de alinear señales y etiquetas.

- **Filas corruptas:** registros con *timestamps* inválidos se descartan filtrando $\text{Timestamp} > 10^9$. (Se encontraron noches con duraciones incluso mayores a 30 h)
- **Exceso de señal o de etiquetas en los bordes:** al inicio o al final de la noche una fuente puede cubrir un rango que las otras no. Como los modelos usan ambas señales, se define la **ventana válida** como la intersección entre el rango etiquetado y el de ambas señales continuas, recortando lo que sobre en cualquiera de los extremos.
- **Gaps internos** (*dropouts* reales del sensor): rompen la continuidad de la secuencia. Se resuelven recortando la cola cuando el prefijo limpio supera el 60 % (*trim_tail*) o descartando la noche entera si el hueco es interior/disperso.
- **Dropouts dispersos de acelerometría:** en vez de descartar la noche completa (demasiado agresivo), se filtra **por época**, descartando sólo las épocas sin cobertura suficiente de ambas señales.

En la Figura 1 se puede visualizar un ejemplo de cómo procesamos las señales de una noche completa.

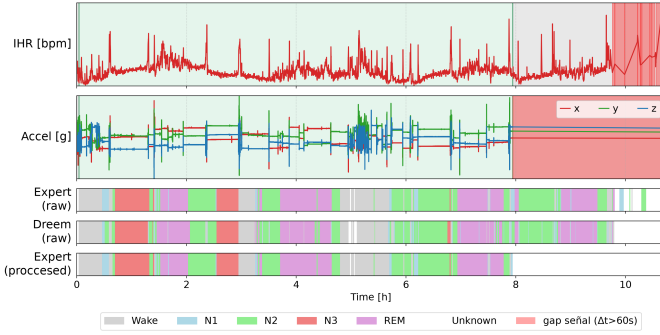


Figura 1. Ejemplo de procesamiento de una noche (Paciente 20, noche 1). En verde la ventana válida —delimitada por el final de la continuidad de la acelerometría— y en rojo la zona problemática. La señal procesada finalmente dura hasta las 7.94 h conservando 949/1282 épocas.

Además, para alimentar a los modelos con la señal cruda tuvimos que llevarla a una representación de tamaño fijo apta como entrada:

Como el IHR y la acelerometría se muestrean de forma irregular, cada época de 30 s se lleva a una grilla fija de 150 *timesteps* (5 Hz, *bins* de 0.2 s), obteniendo un tensor 150×4 . El primer canal es el IHR **interpolado linealmente** sobre la grilla; los otros tres se derivan de la **magnitud** $\|a\| = \sqrt{x^2 + y^2 + z^2}$ (invariante a la orientación del reloj): media y desvío estándar de $\|a\|$ por *bin* y media del ENMO (*Euclidean Norm Minus One*) = $\max(\|a\| - 1, 0)$, que aísla la aceleración por sobre la gravedad. Los *bins* de acelerometría vacíos se rellenan por *forward-fill*. Los cuatro canales se estandarizan con estadísticos de *train*, ya que el IHR domina en magnitud sobre el ENMO.

II-B. Extracción de Características

Para tener un enfoque tabular del conjunto de datos, resumimos cada época de 30 s en un vector de 122 características *handcrafted*, diseñadas a partir del conocimiento del dominio. Se distinguen dos grupos.

Intra-época. De la IHR extraemos estadísticos de nivel y dispersión (media, mediana, desvío, rango intercuartílico y rango) junto con descriptores clásicos de la *variabilidad de la frecuencia cardíaca* (HRV) en el dominio del tiempo y la pendiente de la tendencia dentro de la época. Deliberadamente **no** incluimos las características espectrales de HRV (bandas LF: 0.04 Hz a 0.15 Hz y HF: 0.15 Hz a 0.4 Hz)²; la IHR se muestrea a ≈ 0.2 Hz, por lo que la frecuencia de Nyquist³ (≈ 0.1 Hz) cae por debajo de esas bandas y las vuelve físicamente inobservables. De la acelerometría, calculadas siempre sobre la **magnitud** $\|a\|$ para ser invariantes a la orientación del reloj, obtenemos la media y el desvío del ENMO, la dispersión y el rango de la magnitud, la fracción de inmovilidad (proporción de muestras con $\|a\| \approx 1g$) y la variabilidad del *jerk* (derivada de la magnitud).

Inter-época (contexto temporal). Como la etapa de sueño depende fuertemente de las épocas vecinas, cada característica intra-época se enriquece con su contexto: valores rezagados y adelantados, diferencias respecto de la época previa y estadísticos móviles (media y desvío en una ventana centrada), todo respetando los bordes de cada noche. Se añade además el tiempo transcurrido desde el último movimiento y la posición relativa de la época dentro de la noche.

II-C. Partición y Normalización

Con el fin de evitar *leakage* el conjunto se particiona **por paciente**, quedando 33 sujetos para el entrenamiento y luego 7 para la validación y 7 para el conjunto de test final. Luego ambas representaciones (tabular y señal cruda) se estandarizan con estadísticos de *train*; los modelos densos imputan a la mediana los NaN de borde.

III. METODOLOGÍA

III-A. Métricas de Rendimiento

Dado que se tienen dos etiquetas (expert y dreem) usamos el **coeficiente Cohen's Kappa** (κ) [2], [3] para cuantificar cuán de acuerdo están entre sí, y cuán de acuerdo están nuestros modelos con cada uno de estos.

$$\kappa = \frac{p_o - p_e}{1 - p_e} \quad (1)$$

Donde p_o es la proporción observada de acuerdo (accuracy) y p_e es la proporción de acuerdo esperada por azar.

En la totalidad del dataset, este coeficiente de forma global (todas las clases) entre expert y dreem es de ≈ 0.74 . El objetivo

²Las bandas LF (*Low Frequency*) y HF (*High Frequency*) son rangos de frecuencia del espectro de la HRV asociados a la actividad de los sistemas nerviosos simpático y parasimpático; su relación LF/HF es un indicador clásico del tono autonómico durante el sueño.

³Frecuencia máxima que una señal muestreada puede representar sin *aliasing*: la mitad de la frecuencia de muestreo. Para reconstruir un componente de f Hz hay que muestrear a más de $2f$ Hz.

en principio es intentar lograr un nivel de acuerdo entre la clasificación de nuestro modelo y la clasificación que realiza el especialista en polisomnografía mayor al que tiene con la clasificación automática del dispositivo Dreem.

El desempeño de clasificación se mide principalmente con el $F1_{Macro}$ —promedio del F1 por clase, que al pesar todas las etapas por igual es la métrica más honesta bajo el desbalance— complementado con la **accuracy** global y la **matriz de confusión**, que localiza entre qué etapas ocurren los errores. De forma secundaria reportamos el $F1_w$ (*weighted*, ponderado por la frecuencia de cada clase) únicamente para comparar contra el trabajo de referencia [4], que lo usa como métrica principal.

III-B. Elección de modelos

Como **baseline** utilizamos dos modelos tabulares que hacen uso del feature extraction explicado en la Sección II-B: **XGBoost** y un **perceptrón multicapa (MLP)**.

Los modelos anteriores clasifican cada época de forma aislada, ignorando que el sueño es una secuencia con fuerte estructura temporal —aunque las features fabricadas intentan dar soporte a esta. Para capturarla utilizamos dos modelos que utilizan redes recurrentes:

LSTM bidireccional (BiLSTM) [5], [6] *many-to-many*: procesa la noche completa como secuencia y emite una etapa por época. A diferencia de lo que su nombre podría sugerir, este modelo no opera sobre la señal cruda sino sobre la **representación tabular** de la Sección II-B: la entrada de cada paso x_t es el vector de 122 características de la época t , de modo que la red modela la dinámica *inter-época* sobre esas features. En cada paso t , la celda LSTM regula un estado de memoria c_t mediante tres compuertas —entrada i_t , olvido f_t y salida o_t —:

$$\begin{aligned} (i_t, f_t, o_t) &= \sigma(W_{\{i,f,o\}} x_t + U_{\{i,f,o\}} h_{t-1} + b_{\{i,f,o\}}) \\ \tilde{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\ c_t &= f_t \odot c_{t-1} + i_t \odot \tilde{c}_t, \quad h_t = o_t \odot \tanh(c_t) \end{aligned} \quad (2)$$

donde σ es la función sigmoide y $\odot$ el producto elemento a elemento. La variante **bidireccional** concatena una pasada hacia adelante y otra hacia atrás, $h_t = [\vec{h}_t; \overleftarrow{h}_t]$, de modo que cada época integra contexto pasado y futuro (en *sleep staging offline* se dispone de la noche completa); un cabezal lineal da la predicción por época $\hat{y}_t = \text{softmax}(Wh_t + b)$. La Figura 2 esquematiza esta arquitectura.

Híbrido CNN1D-BiLSTM: (inspirado en los modelos SLAMSS [7]) combina en una sola red dos escalas temporales que los modelos anteriores tratan por separado. Mientras el BiLSTM anterior depende de las características *handcrafted*, aquí prescindimos de ellas y dejamos que la red aprenda su propia representación de cada época a partir de la señal cruda. Consta de dos etapas entrenadas *end-to-end*:

- **Encoder intra-época (CNN 1D)**. Cada época se representa por su tensor crudo 150×4 (Sec. II-A). Una red convolucional 1D —apilando bloques de convolución, normalización y *pooling*— extrae de esa señal un vector

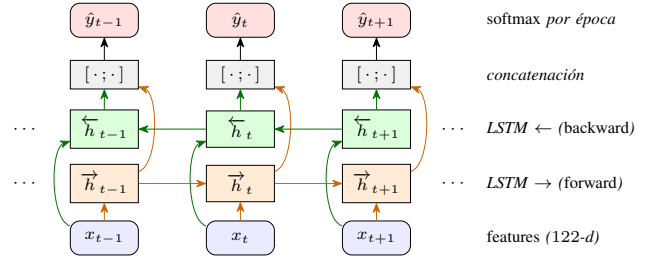


Figura 2. BiLSTM *standalone*: cada vector de características x_t alimenta dos LSTM independientes —una que recorre la noche hacia adelante ($\vec{h}$) y otra hacia atrás ($\overleftarrow{h}$), con parámetros y estados propios que no interactúan entre sí—. En cada época se **concatenan** ambos estados, $h_t = [\vec{h}_t; \overleftarrow{h}_t]$, y un cabezal *softmax* predice la etapa $\hat{y}_t$.

de características x_t , capturando patrones locales dentro de la época que reemplazan a las features diseñadas a mano.

- **Dinámica inter-época (BiLSTM)**. La secuencia de vectores $\{x_t\}$ de toda la noche alimenta la misma BiLSTM *many-to-many* de la Ec. (2), que modela la dependencia entre épocas contiguas y emite una etapa por cada una.

El entrenamiento conjunto es clave: los gradientes del clasificador fluyen hasta la CNN, de modo que la representación intra-época se optimiza específicamente para la tarea de *staging*, en lugar de fijarse de antemano como en las features *handcrafted*. La Figura 3 resume la arquitectura.

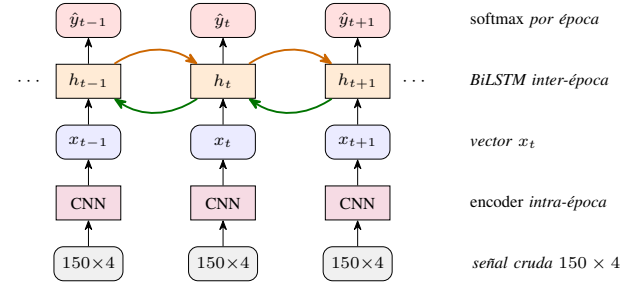


Figura 3. Híbrido CNN1D-BiLSTM: una CNN 1D codifica el tensor crudo 150×4 de cada época en un vector x_t (encoder intra-época); la secuencia $\{x_t\}$ alimenta la BiLSTM (dinámica inter-época) que predice $\hat{y}_t$. Todo se entrena *end-to-end*.

Todos los modelos neuronales se entrenan minimizando la *cross-entropy* ponderada por clase, $\mathcal{L} = -\sum_c w_c y_c \log \hat{y}_c$ con $w_c \propto 1/f_c$, para compensar el desbalance.

IV. RESULTADOS

IV-A. Selección de Hiperparámetros

Los hiperparámetros de los modelos *baseline* se tomaron con valores default de forma de obtener modelos simples y regularizados, mientras que para los modelos secuenciales se ajustan sus hiperparámetros mediante **búsqueda bayesiana**: en lugar de una búsqueda exhaustiva (*grid search*) o aleatoria, se construye un modelo sustituto (*surrogate*) probabilístico de la métrica objetivo en función de los hiperparámetros y, en cada iteración, una función de adquisición propone la

configuración más prometedora, equilibrando exploración y explotación.

Concretamente empleamos el estimador **TPE** (*Tree-structured Parzen Estimator*) [8], que a partir de los *trials* ya evaluados separa las configuraciones en *buenas* y *malas* según un umbral de la métrica y estima con ventanas de Parzen una densidad para cada grupo, $l(x)$ (buenas) y $g(x)$ (malas); la próxima configuración a probar se elige maximizando la razón $l(x)/g(x)$. Su estructura de árbol permite además manejar hiperparámetros condicionales.

La implementación se realiza con la librería **Optuna** [9], que provee el muestreador TPE junto con *pruning* de *trials* poco prometedores para abaratar el costo de la búsqueda. El objetivo optimizado es el **Cohen’s Kappa** sobre una *única* partición de *validación* —por límites computacionales no recurrimos a *cross-validation*, dado el alto costo de entrenar cada modelo secuencial (en particular el híbrido, que corre una CNN por época de cada noche)— y la configuración ganadora se reentrena sobre *train+val* y se reporta una única vez sobre *test*. El espacio de búsqueda por modelo, el número de iteraciones y los valores finales seleccionados se detallan en el Cuadro I.

IV-B. Clasificación

El *baseline* XGBoost alcanza $F1_{Macro} \approx 0.44$ y $\kappa \approx 0.32$ (Ec. (1)) frente al experto, con *accuracy* de $\approx 51\%$ (ver Cuadro I). Pese al desbalance, capta señal discriminativa en las cinco clases, aunque lejos del techo humano ($\kappa \approx 0.74$); los errores se concentran entre etapas adyacentes (N_1 vs. N_2 y REM). En particular, N_1 —transitoria y minoritaria— resulta casi indetectable para un clasificador por época: su firma en IHR y movimiento apenas se distingue de N_2 o *Wake*, de modo que, sin una dependencia temporal explícita, el modelo no logra aislarla.

Cuadro I
RENDIMIENTO EN TEST (5 CLASES) FRENTE AL ETIQUETADO DEL EXPERTO. $F1_M$: F1 MACRO; $F1_w$: F1 *weighted*; κ : COHEN’S KAPPA; ACC.: *accuracy*.

Modelo	$F1_M$	$F1_w$	κ	Acc.
Dreem	0.7076	0.7839	0.7173	0.7924
XGBoost	0.4450	0.4912	0.3244	0.5063
MLP	0.4515	0.5044	0.3475	0.5253
BiLSTM	0.5673	0.5932	0.4630	0.5880
Híbrido	0.4773	0.4767	0.3570	0.4795

Los dos *baselines* tabulares rinden de forma equivalente (XGBoost $\kappa \approx 0.32$, MLP $\kappa \approx 0.35$): la red densa no mejora a los árboles sobre las mismas *features*. La mayor mejora la aporta la **BiLSTM** ($F1_{Macro} \approx 0.57$, $\kappa \approx 0.46$), ~ 12 puntos de κ por encima, lo que confirma que el **contexto inter-época** es el factor determinante. El **híbrido** ($\kappa \approx 0.36$) se ubica por debajo pese a operar sobre la señal cruda: su *encoder* convolucional debe aprender de cero un extractor de características intra-época sobre un número acotado de noches y sin preentrenamiento, lo que lo vuelve propenso al sobreajuste y explica que la señal cruda no supere a las *features* diseñadas a mano. Todos los modelos permanecen lejos de la

cota de referencia (Dreem, $\kappa \approx 0.72$): la brecha —limitación de las señales de muñeca frente al EEG— se concentra en N_1 y en las transiciones entre etapas contiguas. A continuación se presenta una segunda evaluación con 4 clases, agrupando N_1 y N_2 en *Light* y N_3 en *Deep*, por la similitud fisiológica entre esas etapas y para simplificar la tarea de clasificación.

Cuadro II
RENDIMIENTO EN TEST (4 CLASES) FRENTE AL ETIQUETADO DEL EXPERTO.

Modelo	$F1_M$	$F1_w$	κ	Acc.
Dreem	0.8457	0.8672	0.8009	0.8669
XGBoost	0.5538	0.5496	0.3363	0.5419
MLP	0.5681	0.5669	0.3612	0.5600
BiLSTM	0.6500	0.6509	0.4907	0.6519
Híbrido	0.5666	0.5684	0.3927	0.5690

Al agrupar en 4 clases todas las métricas mejoran, dado que se elimina la confusión N_1-N_2 , la más frecuente. El ordenamiento entre modelos se mantiene: la **BiLSTM** continúa siendo el modelo de mejor desempeño ($F1_{Macro} \approx 0.65$, $\kappa \approx 0.49$) y el híbrido permanece por debajo, lo que confirma que la ventaja del contexto inter-época no depende de la granularidad del *target*. La brecha con la cota de referencia (Dreem, $\kappa \approx 0.80$) se reduce respecto de las 5 clases pero persiste, lo que evidencia que la limitación subyacente son las señales de muñeca y no la dificultad de separar N_1 de N_2 .

Incorporamos además el $F1_w$ (F1 *weighted*), la métrica principal reportada por el trabajo de referencia [4], que al ponderar cada clase por su frecuencia resulta más representativa bajo el fuerte desbalance de este dataset. Como premisa el buen desempeño en las clases mayoritarias (N_2 , REM), $F1_w$ es sistemáticamente superior al $F1_{Macro}$: la BiLSTM alcanza ≈ 0.59 en 5 clases y ≈ 0.65 en 4. Medidos con este criterio, nuestros modelos **se aproximan de forma razonable** al desempeño reportado en dicho trabajo, pese a prescindir del EEG y operar únicamente con señales de muñeca; la distancia con el $F1_w$ del experto–Dreem ($\approx 0.78/0.87$) es sensiblemente menor que la sugerida por el $F1_{Macro}$, que penaliza con dureza la clase N_1 .

Como complemento a las métricas de decisión, el Cuadro III reporta el promedio macro (one-vs-rest, 5 clases) del área bajo las curvas ROC (AUC) y Precision-Recall (AP), que evalúan la calidad del *ranking* de probabilidades de cada modelo con independencia del umbral de decisión.

Cuadro III
PROMEDIO MACRO (ONE-VS-REST, 5 CLASES) DE LAS CURVAS ROC Y PRECISION-RECALL EN TEST.

Curva	XGBoost	MLP	BiLSTM	Híbrido
AUC-ROC macro	0.772	0.775	0.843	0.829
AP (PR) macro	0.488	0.497	0.603	0.569

Ambas curvas reproducen el mismo ordenamiento observado en las métricas de decisión: la BiLSTM lidera (AUC = 0.843, AP = 0.603), seguida del híbrido, con los *baselines* tabulares por detrás. La AP, más informativa bajo desbalance, se mantiene sensiblemente por debajo de la AUC en todos los modelos, lo que refleja la dificultad persistente en las

clases minoritarias. Que la BiLSTM supere a los baselines también en la calidad del *ranking* —y no sólo en la decisión final— indica que el contexto inter-época mejora la confianza asignada a cada etapa, no únicamente el umbral de corte. La Figura 4 reúne las matrices de confusión (4 clases) de los cuatro modelos.

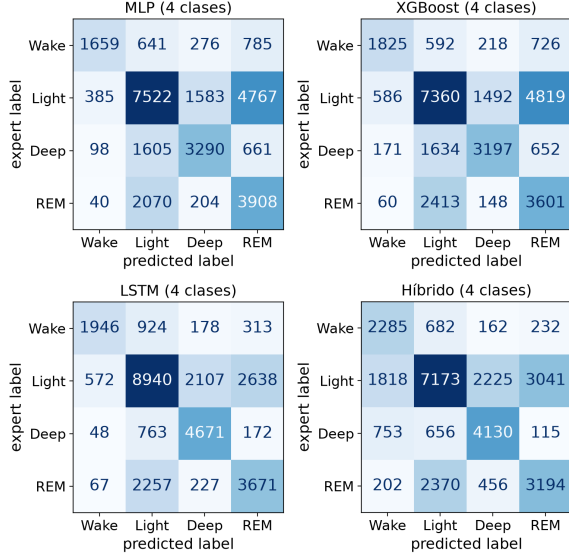


Figura 4. Matrices de confusión (4 clases, test, vs. experto). Arriba, los *baselines* MLP (izq.) y XGBoost (der.); abajo, los secuenciales BiLSTM (izq.) e híbrido (der.).

En los *baselines* (fila superior) el error dominante es la confusión *Light*↔*REM*, agravada por una sobre-predicción de *REM* y *Wake* inducida por la ponderación de clases. Este comportamiento es esperable: sin EEG, el *REM* —IHR irregular y cuerpo inmóvil— es casi indistinguible del sueño ligero para modelos que sólo observan proxies autonómicos y de movimiento, y al clasificar cada época de forma aislada no disponen del contexto que desambiguaría esas etapas. Los modelos secuenciales (fila inferior) marcan más la diagonal justamente porque el contexto inter-época aporta las regularidades de transición que faltan: la **BiLSTM** es la más limpia —recupera mejor *Light* y *Deep*, con el error residual en la frontera *Light*–*REM*. El **híbrido** sobre-predice *Wake* y recupera menos *Light*: su *encoder* convolucional, entrenado de cero sobre pocas noches, aprende una representación intra-época ruidosa que arrastra a la BiLSTM. En todos los casos *Deep* es la clase mejor resuelta, por su firma fisiológica más nítida (IHR mínimo e inmovilidad).

Finalmente, la Figura 5 muestra un resultado cualitativo: los hipnogramas predichos para una noche de test superpuestos a los de *expert* y *dreem*.

La BiLSTM reproduce la macroestructura del hipnograma del experto —los bloques prolongados de *N2/Deep* y los episodios *REM* hacia el final de la noche—, concentrando sus errores en las transiciones entre etapas. El híbrido, en cambio, genera una secuencia más fragmentada y con más falsos *Wake*,

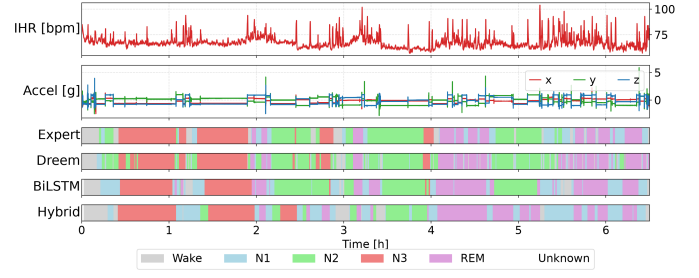


Figura 5. Datos de una noche (Paciente 31, Noche 1) junto a las etiquetas *expert* y *dreem* y las clasificaciones de los modelos secuenciales **BiLSTM** e **Híbrido**.

lo que ilustra a nivel de una sola noche la brecha de desempeño que reflejan las métricas agregadas.

V. CONCLUSIÓN

Presentamos un *pipeline* completo de estadificación del sueño a partir de señales de muñeca (IHR y acelerometría), desde el control de calidad hasta una progresión de modelos que va del *baseline* tabular a arquitecturas secuenciales. El mejor desempeño lo obtuvo la **BiLSTM** sobre *features hand-crafted* ($F1_{Macro} \approx 0.57$, $\kappa \approx 0.46$ en 5 clases; $\kappa \approx 0.49$ en 4), superando a los *baselines* tabulares (XGBoost y MLP, $\kappa \approx 0.32$ – 0.35) y al híbrido CNN–BiLSTM ($\kappa \approx 0.36$). Esto confirma el eje del trabajo: el factor determinante es el **contexto temporal inter-época** por encima de la representación intra-época o del clasificador. El híbrido, pese a operar sobre la señal cruda, no superó al enfoque tabular por el sobreajuste de su *encoder* entrenado de cero. Aun así, todos los modelos quedan por debajo del Dreem (acuerdo experto–Dreem, $\kappa \approx 0.72$), una brecha que refleja el límite intrínseco de las señales de muñeca frente al EEG, concentrada en N_1 y en las transiciones entre etapas contiguas.

El trabajo futuro se articula en los dos niveles de representación. A nivel **intra-época**, el bajo desempeño del aprendizaje *end-to-end* sugiere reemplazar el *encoder* convolucional supervisado por un **preentrenamiento auto-supervisado** (p. ej. *autoencoders* por reconstrucción, sin etiquetas) que induzca una *embedding* más robusto y mitigue el sobreajuste. A nivel **inter-época**, siendo la dependencia entre épocas el factor dominante, resulta natural sustituir la recurrencia de la BiLSTM por **auto-atención (Transformers)**, capaz de capturar dependencias de largo alcance sin la limitación del gradiente secuencial. En ambas direcciones, la referencia a seguir es **SLAMSS-IFS** [4], el modelo *sequence-to-sequence* del trabajo asociado a este mismo dataset —que supera a nuestra BiLSTM en apenas ~5% de accuracy (4 clases)—, combinando ambas mejoras; profundizar en esa línea acercaría nuestro desempeño al estado del arte.

APÉNDICE A INTERPRETABILIDAD DE LAS FEATURES (SHAP)

La Figura 6 muestra la importancia de las *features* a partir del modelo de XGBoost implementado medida con **valores SHAP** [10] —más fiable que el *gain* nativo, sesgado ante *features* correlacionadas—, desglosada por clase. El resultado es revelador: las variables más influyentes son casi todas de **contexto temporal** —la posición de la época en la noche (*epoch_frac*), estadísticos móviles (*_rstd/_rmean*) y el tiempo desde el último movimiento (*epochs_since_move*)—, por encima de cualquier descriptor puramente intra-época. Es decir, incluso el modelo tabular apoya su decisión en la estructura temporal del sueño, lo que anticipa la ventaja de los modelos secuenciales. El fuerte peso de *epoch_frac* refleja que la arquitectura del sueño es predecible a nivel poblacional (el N_3 domina el primer tercio de la noche y el REM el último), aunque apoyarse en la posición conlleva el riesgo de peor generalización a noches atípicas.

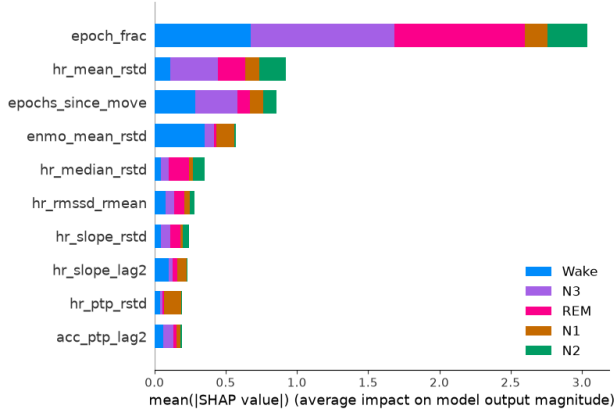


Figura 6. Importancia de *features* del XGBoost por valores SHAP (*mean|SHAP|*), desglosada por etapa. Las variables dominantes son de contexto temporal inter-época (*epoch_frac*, *_rstd*, *epochs_since_move*).

APÉNDICE B CNN 1D INTRA-ÉPOCA (*standalone*)

Entrenamos la CNN 1D de forma aislada —sin la BiLSTM— para clasificar cada época directamente desde su tensor crudo 150×4 , con el fin de aislar el aporte *intra-época*. El resultado es contundente: al carecer de contexto entre épocas, la señal cruda por sí sola rinde a un nivel **cercano al azar** ($\kappa \approx 0.04$, $F1_{Macro} \approx 0.22$, $accuracy \approx 0.26$ en 5 clases; apenas $\kappa \approx 0.04$ en 4). Esto confirma que la información *intra-época* es, por sí misma, insuficiente para el *staging*, y es coherente con que el híbrido —que reutiliza esta CNN como *encoder*— tampoco supere al enfoque tabular: sólo al combinarla con la dinámica *inter-época* la señal cruda aporta información útil.

APÉNDICE C AUTOENCODER LSTM (REPRESENTACIÓN NO SUPERVISADA)

Un *encoder* BiLSTM comprime cada noche en un *embedding* por época z_t ($L = 32$) y un *decoder* LSTM reconstruye las características; se entrena **sin etiquetas**, minimizando el error de reconstrucción. La pregunta de investigación es si una representación aprendida de forma no supervisada captura estructura útil para el *staging* en comparación con las *features* diseñadas a mano. Los *embeddings* z_t se emplean luego como entrada de un XGBoost. La Figura 7 esquematiza la arquitectura.

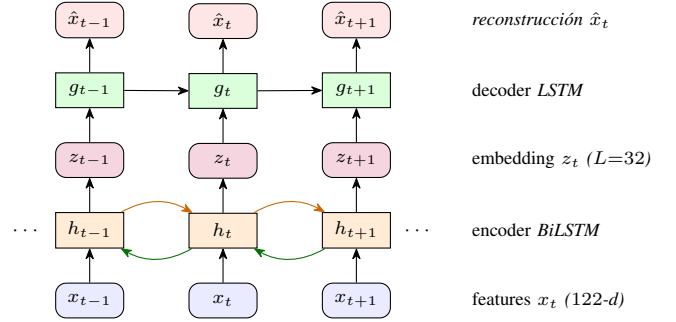


Figura 7. Autoencoder LSTM: el *encoder* BiLSTM comprime la secuencia de *features* de la noche en un *embedding* por época z_t ($L=32$) y el *decoder* LSTM reconstruye $\hat{x}_t$. Se entrena sin etiquetas minimizando $\|x_t - \hat{x}_t\|$; el *bottleneck* z_t es la representación que alimenta al XGBoost.

APÉNDICE D ANÁLISIS CONTROLADO DE LOS EMBEDDINGS

Comparamos, sobre un **único split**, un XGBoost con tres entradas: sólo *handcrafted* (HC), sólo embeddings (EMB) —generados por el autoencoder del Apéndice C— y la concatenación de ambas (ALL) (Cuadro IV). Los *embeddings* por sí solos quedan por debajo de las *handcrafted* ($\Delta\kappa \approx -0.09$) y sumarlos a estas no aporta mejora ($\Delta\kappa \approx -0.001$); además, la importancia por ganancia del XGBoost está dominada por las *features* manuales. Concluimos que, en esta tarea, la representación no supervisada no supera al *feature engineering* basado en conocimiento del dominio.

Cuadro IV
XGBOOST CON DISTINTAS ENTRADAS SOBRE EL MISMO *split* (TEST, VS. EXPERTO). HC: *handcrafted*; EMB: EMBEDDINGS DEL AUTOENCODER; ALL: AMBAS.

Entrada	$F1_M$	κ (5 cl.)	κ (4 cl.)
HC (122)	0.4707	0.3776	0.3865
EMB (32)	0.3971	0.2867	0.2919
ALL (154)	0.4686	0.3762	0.3847

REFERENCIAS

- [1] T.-A. Song, “A Multi-Night Instantaneous Heart Rate and Accelerometry Dataset with EEG Sleep Stage Labels,” *PhysioNet*, May 2026, version 1.0.0. [Online]. Available: <https://doi.org/10.13026/a0sy-7t69>
- [2] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

- [3] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.
- [4] T.-A. Song, Y. Zhang, Z. Zhou, L. Hou, M. Malekzadeh, A. Behzad, and J. Dutta, "Ai-driven sleep staging using instantaneous heart rate and accelerometry: Insights from an apple watch study," *IEEE Transactions on Biomedical Engineering*, vol. 73, no. 4, pp. 1596–1608, 2026.
- [5] S. Hochreiter and J. Schmidhuber, "Long short-term memory," *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [6] M. Schuster and K. Paliwal, "Bidirectional recurrent neural networks," *IEEE Transactions on Signal Processing*, vol. 45, no. 11, pp. 2673–2681, 1997.
- [7] T.-A. Song, S. R. Chowdhury, M. Malekzadeh, S. Harrison, T. B. Hoge, S. Redline, K. L. Stone, R. Saxena, S. M. Purcell, and J. Dutta, "Ai-driven sleep staging from actigraphy and heart rate," *PLOS ONE*, vol. 18, no. 5, pp. 1–29, 05 2023. [Online]. Available: <https://doi.org/10.1371/journal.pone.0285703>
- [8] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for hyperparameter optimization," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 24, 2011.
- [9] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," *CoRR*, vol. abs/1907.10902, 2019. [Online]. Available: <http://arxiv.org/abs/1907.10902>
- [10] S. M. Lundberg and S. Lee, "A unified approach to interpreting model predictions," *CoRR*, vol. abs/1705.07874, 2017. [Online]. Available: <http://arxiv.org/abs/1705.07874>
- [11] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Köpf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, "Pytorch: An imperative style, high-performance deep learning library," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 32, 2019, pp. 8024–8035.
- [12] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg, J. Vanderplas, A. Passos, D. Cournapeau, M. Brucher, M. Perrot, and E. Duchesnay, "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [13] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785–794.
- [14] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith *et al.*, "Array programming with NumPy," *Nature*, vol. 585, no. 7825, pp. 357–362, 2020.
- [15] W. McKinney, "Data structures for statistical computing in Python," in *Proceedings of the 9th Python in Science Conference (SciPy)*, 2010, pp. 56–61.
- [16] J. D. Hunter, "Matplotlib: A 2d graphics environment," *Computing in Science & Engineering*, vol. 9, no. 3, pp. 90–95, 2007.